COSC3043 - Data Science Project – Final Report - Report

AI First Nations – Data Science Project Final Report

Students:

Kirtan Shah – S4108558

Thomas Blanks – S4094959

Jake Dyl – S4092833

Industry Supervisor: John Fejo (john.aifirstnations@gmail.com)

Academic Supervisor: Damiano Spina (damiano.spina@rmit.edu.au)

Acknowledgement of Country....3
Introduction....3
Scope Change....3
Approach & Methodology for Deliverables....3
Methodology:....4
Evaluation Framework....4
Binary Classifier (Kaggle)....4
Binary Classifier (EfficientNet)....6
Evaluation & Preference....8
Multi-class Classification Model (Mangrove Species)....8
Flask Application....9
Model and Flask App Integration....11
Baseline Random Forest Binary Classification Model Evaluation....12
Advanced Custom Neural Network Binary Classification Model Evaluation....12
Advanced Custom Neural Network Image Classification Model Evaluation....13
Results and Findings....14
Conclusion....15
References....16
Appendices....17
Code and Project Management....17
Roles and Responsibilities....17
Self-reflection....17
Kirtan....17
Thomas....18
Jake....18

Acknowledgement of Country

We acknowledge the people and the unceded lands of the Woi wurrung, Boon wurrung, and Wathaurong language groups of the Kulin Nation upon which the task of training, testing and deployment of our machine learning model was conducted.

We would also like to acknowledge the people of the Bagirbarra and Larrakia clan groups on whose land data was collected for the purpose of training our machine learning models.

We pay our respects to Elders, past, present and emerging, with hope that knowledge systems continue to grow in sophistication, augmented with technological advancements made in partnership by Indigenous and non-Indigenous parties.

Introduction

AI First Nations is an Indigenous-led startup supported by KJR. Their work primarily looks at leveraging Data Science and Machine Learning techniques to augment existing drone-imagery pipelines.

The initial aims of this project were to develop a model that could generate counts of mangrove trees of different species from orthomosaic maps provided by AI First Nations. The deliverables on this project were both a machine learning model as well as a locally hosted Flask web application to provide an easy-to-use interface to run the analysis model through.

Scope Change

The scope outlined in the initial project proposal was unfortunately adjusted in response to unforeseen changes in circumstances causing delays in data provision.

While waiting for the data to come in, we took time to create a Flask application and a prototype model (our binary Random Forest Classifier for mangrove vs. non-mangrove detection) using satellite data as a proof of concept to present to the AI First Nations team.

We also decided to shift our focus to creating a simple and approachable interface to allow users to upload images to the backend for analysis, creating a system that could classify mangroves seen in an image by species (yellow, red, or orange).

This was all achieved, meaning our work can serve as a solid foundation for further development in the future. We took care to write extensive documentation and instructions for our code, which was also structured in a highly modular and scalable manner to support future development.

We were also able to create an early prototype for the tree-counting algorithm, however we were unable to get it in an acceptable state to be fully implemented in time, so this will not be included in the final project showcase.

Approach & Methodology for Deliverables

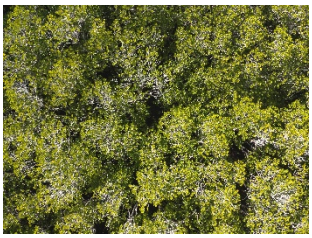
A Kanban-Agile methodology was utilised throughout the duration of the project. Tools such as Trello allowed for greater organisation and transparency for both the academic and industry-partner supervisors, who were able to use these tools to oversee and monitor our progress. With weekly sprints, both academic and industry-partner supervisors were kept informed on our successes and pain points as we worked to achieve our goals. Support on both fronts gave the team a greater level of clarity and enabled us to work effectively together and overcome challenges.

From the project proposal, two primary tasks were identified. As discussed in the aim a pre-trained classification machine learning model would have to be developed, as well as an interface such as a Flask web framework so that data can be submitted, analysed and displayed.

To do this, a dataset of 477 aerial mangrove images (83 red, 156 orange, and 238 yellow mangroves) was provided for training and development of a classification model, examples are included in *Table 1*. The data is sourced from Darwin, Northern Territory, Australia and Innisfail, Queensland, Australia.

Earlier in the pipeline, models for binary classification were also created for the purpose of predicting whether provided images contained mangroves.

Table 1 Examples of yellow, orange and red mangroves



Yellow



Orange



Red

Methodology:

We set up a pipeline with two machine learning models for image analysis. The first was a binary classifier to detect whether an image contained a mangrove canopy. If a mangrove canopy was detected, the image would then be passed on to the multi-class classification model to predict the species class of the mangrove imagery. The result of this analysis would then be shown to the user alongside metrics such as the model's confidence in its prediction in the Flask web application.

Evaluation Framework

The evaluation framework to evaluate the machine learning models included metrics such as F1-score, precision, recall and accuracy. These features all contributed to the overall accuracy and precision of the models by providing greater insight into the models and where they were lacking.

The evaluation metrics used in the models are shown and defined as per our needs in *Table 2*.

Table 2 Evaluation metrics used for evaluation through model development

Metric	**Rationale**	**Formula**
F1-score	Provides an amalgamated metric, combining precision and recall. It is particularly helpful when classes are imbalanced within the dataset [1].	$F_1\ score = 2 * \frac{Precision * Recall}{Precision + Recall}$
Precision	Represents the proportion of true positive predictions [2].	$Precision = \frac{True\ Positives}{True\ Positives + False\ Positives}$
Recall	Represents the proportion of actual positive predictions correctly identified [2].	$Recall = \frac{True\ Positives}{True\ Positives + False\ Negatives}$
Accuracy	Represents the proportion of correct predictions out of the entire dataset. Not recommended for imbalanced datasets [3].	$Accuracy = \frac{True\ Positives + True\ Negatives}{All\ Predictions}$

Binary Classifier (Kaggle)

While waiting for drone imagery of the mangrove canopies from AI First Nations, we developed a binary classifier based on data sourced from Kaggle [4] to present as a proof-of-concept to our industry supervisor. This allowed us to test the model and Flask integration early, so that when we

received the data for our multi-class classifier, we already had the code required to integrate it into the Flask application.

Both binary classifiers issued predictions from inputted imagery, assigning a 1 for present mangrove canopies, and a 0 if none were detected. Two approaches were considered: using a random forest classifier from the `sklearn.ensemble` module, and another using EfficientNet convolutional neural network.

The random forest classifier method was selected for its ensemble training process. This enabled minimal data processing, creating less opportunity for overfitting, and thus less bias by building multiple 'forests' and providing more accurate prediction.

Exploratory Data Analysis

First, a function was used to view a sample of both mangrove and non-mangrove imagery in the Kaggle dataset (seen below). A large portion of the images were corrupted and did not contain enough information to conduct training.

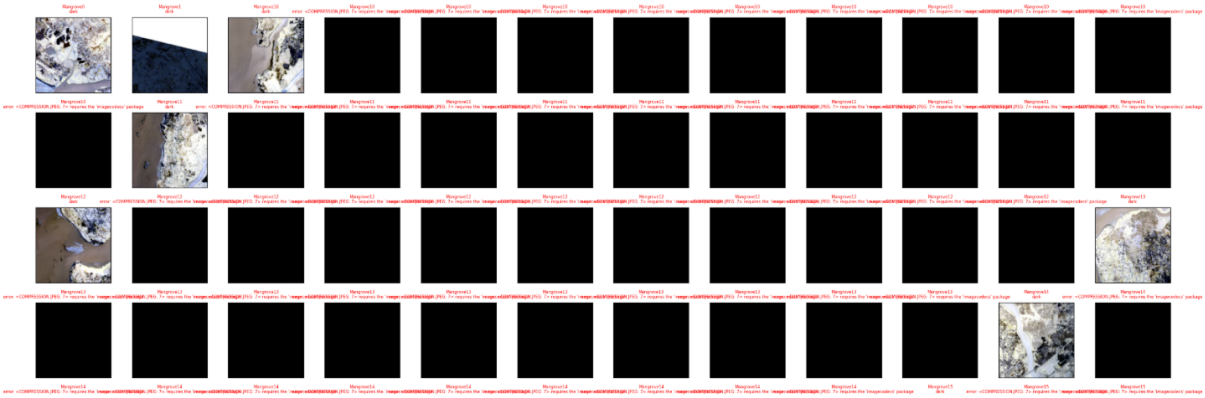


Figure 1 Randomly selected sample mangrove imagery

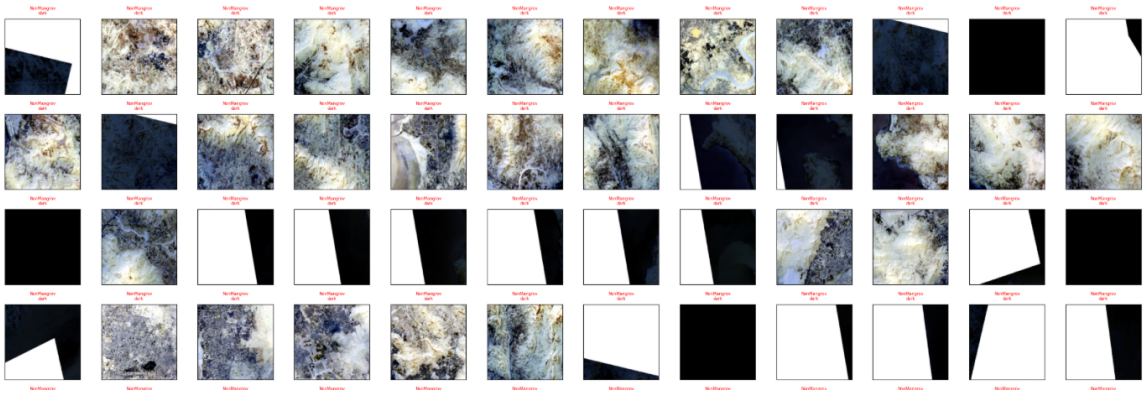


Figure 2 Randomly selected sample non-mangrove imagery

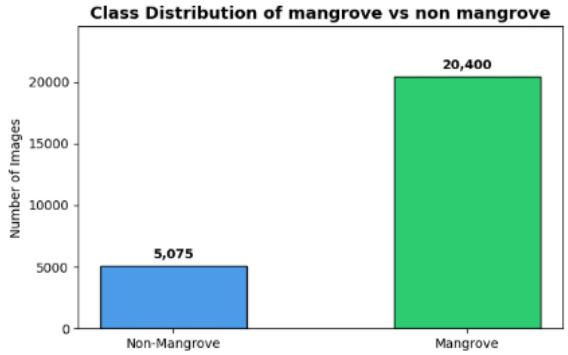


Figure 3 Class distributions of mangrove and non-mangrove subjects within the dataset. As can be observed from the graph, there is a significant imbalance, with 20,400 mangrove images and 5,075 non-mangrove images

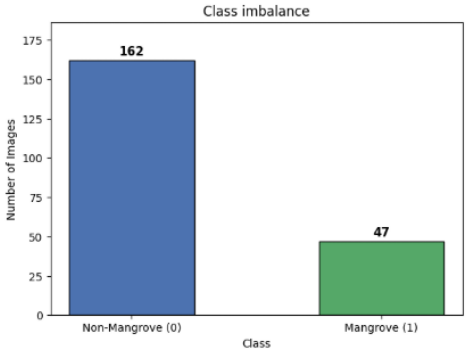


Figure 4 Class counts after pre-processing; as can be seen in the numbers, there was a significant decrease in images for either class due to data corruption, non-viability for training, etc.

This indicated a severe class imbalance within the dataset which would have to be rectified or at least dealt with later in the training process. This was exacerbated in an inverse fashion once pre-processing had been completed.

Pre-processing

Pre-processing in the tree-based classifier involved the removal of corrupted and non-valid image files to avoid the inclusion of junk data in the set which could harm the efficacy of the model in identifying mangrove and non-mangrove imagery. This involved the TIFF files from the dataset were loaded into the program to be converted into arrays of 256x256 pixels with 7 spectral bands (256x256x7) and classified as either 'valid', 'dark', 'uniform', or 'error'.

Once this had been completed, statistical and qualitative features were extracted from the data using defined methods in the program. One method looked at the statistical features of each array, such as their means, their standard deviations, minima and maxima, as well as their interquartile ranges. Another created a grey level co-occurrence matrix, observing variations to observe textural features within the image. After this was completed, the data were split into training, testing and validation sets.

Training

Once pre-processing was complete, the models were built.

The random forest binary classifier was built using the `sklearn.pipeline` module. This is used to allow a series of transformers to be applied. In our case, a `StandardScaler` was applied to normalise the data before being entered into the random forest classifier for training. In training, no max-depth parameter was set, the number of estimators was set to 200, using the Gini criterion. The `min_samples_split` was set to 4, this decreased the likelihood of the model overfitting.

Binary Classifier (EfficientNet)

The neural network-based model was chosen for the pre-built format, preserving the weights from the EfficientNet-B0 base model, as well as the spatial relationships in images.

Exploratory Data Analysis

This model was created later than the RandomForestClassifier, and thus we were able to leverage the use of the actual mangrove imagery from AI First Nations. However, this came with the caveat that only mangrove imagery was provided, rather than non-mangrove imagery. This meant that non-mangrove imagery had to be sourced from other datasets across sites like Kaggle [5, 6, 7, 8, 9]. Furthermore, the sourcing of new datasets was to expand the verity of image types present within the non-mangrove dataset. Due to the surplus of green non-mangrove images, the model incorrectly

generalised and predicted images. By adding more verity in scenic images the model could generalise non-mangroves more efficiently.

Pre-processing

Pre-processing in the EfficientNet classifier took a similar, but ultimately different shape to the random forest classifier. Here, all the files within the requisite folders were selected, labelled with a 1 (meaning mangrove), or a 0 (meaning non-mangrove). Once this was completed the images were augmented and normalised using transformations and augmentations from the `transform's` module from `torchvision`. This meant that all the files were resized to 224 by 224 pixels, and that random flips, rotations, colour changes were conduct on images in the set and then converting images to tensors to use in training.

Once tensors had been created, they were assigned class-based weights to handle the class imbalance. This was calculated as the inverse of their count in the dataset, and then randomly sampled before splitting into training, testing and validation sets.

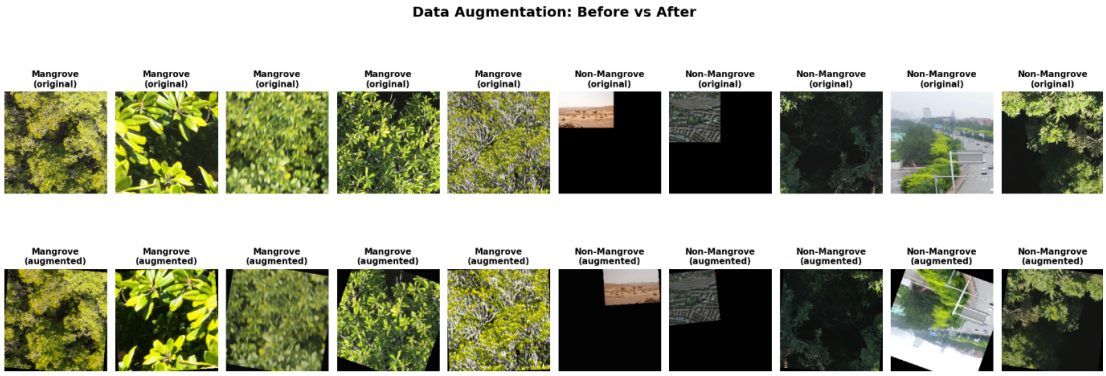


Figure 5 Results of data augmentations and transformations

Training

EfficientNet-B0 was chosen as the base architecture for the neural network-based binary classifier, using trained weights derived from IMAGENET1K_V1. This ensured that latent spatial relationships are kept and built on with further training. CNNs, particularly EfficientNet, were chosen for the model in our approach as they provided an efficient and accurate model to augment the binary classification approach as outlined in Tan and Le's paper proposing the EfficientNet architecture [10].

Before training could begin the model had to be adapted to the task. The images provided were 4000x3000 pixels and would be expected to contain RGB colouring. This meant that inputs would first have to be tiled into 512x512 pixel arrays to be used as inputs for the model to predict on.

Once this was complete, the base model was imported, and all its layers were frozen. The model classifier was built atop this. This component of the training process demonstrated a similarly sequential process to the random forest method, starting with a dropout function to probabilistically shut down layers of the model to isolate small enclaves of networks within the model to allow them to develop independently, lowering the dependency on specific neurons, thus improving generalisability of the model in the long run. The classifier end of the model was replaced with a 2-class output, and then the model was moved to the GPU.

Training epochs ran with a forward pass, calculating loss, backward propagation, and an update, this ran with an Adam optimiser. Adam optimiser was chosen as it performs better on sparse data [11]. Once optimisation and updates are conducted, the performance was evaluated using the `validationEpoch` method to check its performance.

These methods were implemented in a training loop, saving the model each time the model improves. The training loop used an early stop triggered if no improvements were indicated after five loops.

Evaluation & Preference

The evaluation metrics used in this methodology were the F1-score (macro-average), precision, recall, accuracy and confusion matrices. These provided nuance and stronger understandings of challenges and strengths of the models.

As the EfficientNet model dealt with the class imbalance by applying weights and accounting for weights in the sampler prior to splitting, this provided a more accurate and precise insights into the model. This is further discussed in the results section.

Multi-class Classification Model (Mangrove Species)

Three classes were identified as the targets (yellow, orange, and red mangroves). The data provided for training were JPEG files containing 3 spectral bands (RGB), these images had already been labelled, also making this task a supervised learning task.

The supervised classification model leveraged a convolutional neural network (CNN) architecture utilising Google's pre-trained EfficientNet-B0 model as a base for feature extraction. EfficientNet was selected for its ability to encapsulate and extract key spatial features of images while maintaining a relationship using neural networks.

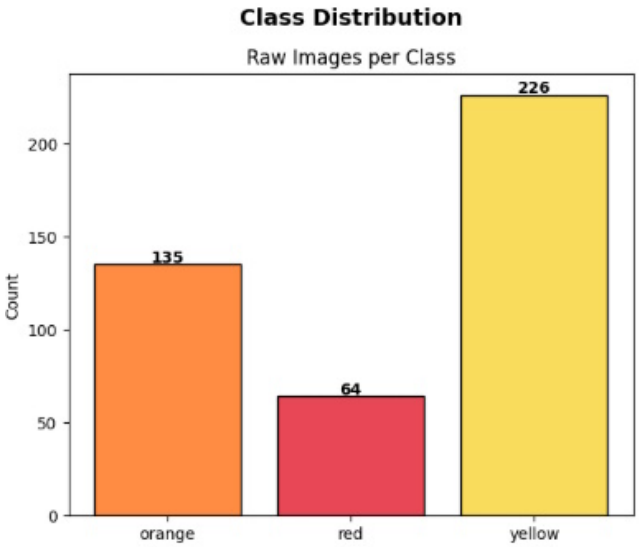


Figure 6 Class counts of each species

EfficientNet uses compound scaling, this meant we could build deep neural networks resulting in strong and efficient performance. The model was built and trained on 425 images each split into their respective classes (*Figure 6*).

Using weighted sampling to address the class imbalance as illustrated in *Figure 6*, the model could then leverage adequate multi-scale feature extraction within EfficientNet to classify mangrove species.

Data augmentation was applied within the dataset to expand the variety of mangrove images attempting to reduce overfitting from class imbalance. This decreased the constraint of limited data.

By transforming, rotating, flipping and applying colour attribute changes onto the training data, we increased the total class size, further helping close the class imbalance.

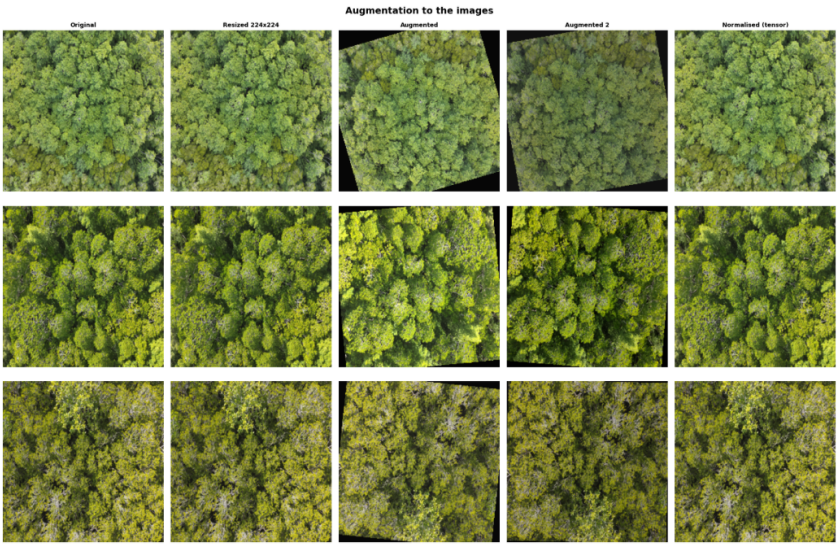


Figure 7 Image augmentations on five separate samples.

For the species classification model, a training split of 70% training, 15% validation and 15% was chosen. The split offered a balance between the model learning from training data while retaining enough data to tune hyper-parameters. The testing set was used to analyse the tuning and overall model performance. The model was optimized by using dropout and early stopping through the learning loop as discussed earlier in the binary classifier.

Model performance was evaluated using a classification report containing a variety of metrics such as F1-scores, precision, recall, and accuracy (discussed in the Evaluation Framework section). These metrics allowed for greater insight into the performance of the model. Confusion matrices were also created for evaluation purposes. These provided a heuristic as to the performance of the model based on the number of true positives, true negatives, false positives, and false negatives.

Flask Application

As mentioned, one of our goals was to integrate our machine learning models into an easy-to-use interface for in the field. This made Flask an attractive option for us, for its high level of customisability with HTML and CSS, ability to be locally hosted on devices, and accessible on familiar browser platforms. The ability to natively integrate Python machine learning models by creating external modules which could be called into the program was also a key part of our consideration of it.

The Flask application contains HTML templates for each page. Currently, there are two different pages supported, the home page and the result analysis page.

The home page provides users with information about AI First Nations as a company, their overarching goal with the technology, as well as user instructions for how to upload their image for machine learning analysis. When pressing the browse button, the application will prompt users to

upload a .png or .jpeg/jpg file. This will send the file to the ‘uploads’ directory in the backend when the ‘Analyse Image’ button is clicked.

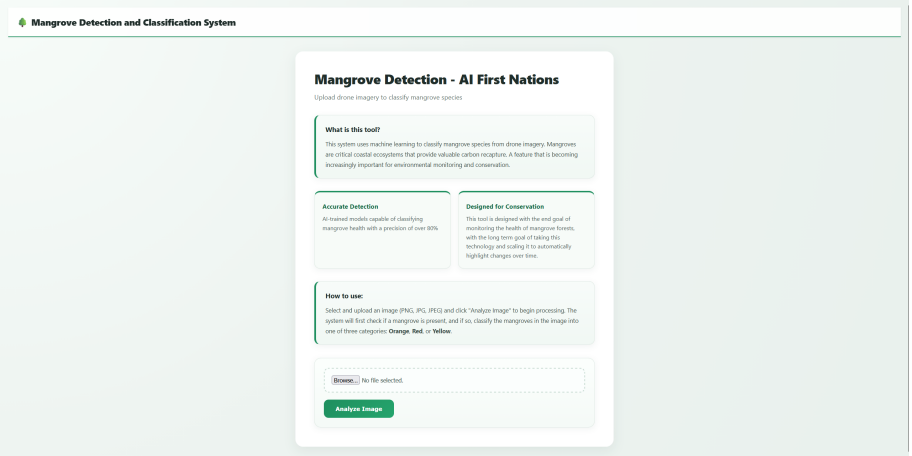


Figure 8 Dashboard interface for image upload

After clicking the button, the user will be redirected to the Analysis Results page, this is where they will receive the results of the analysis.

If the binary classifier did not determine that the image contains a mangrove, the following error message will be shown.

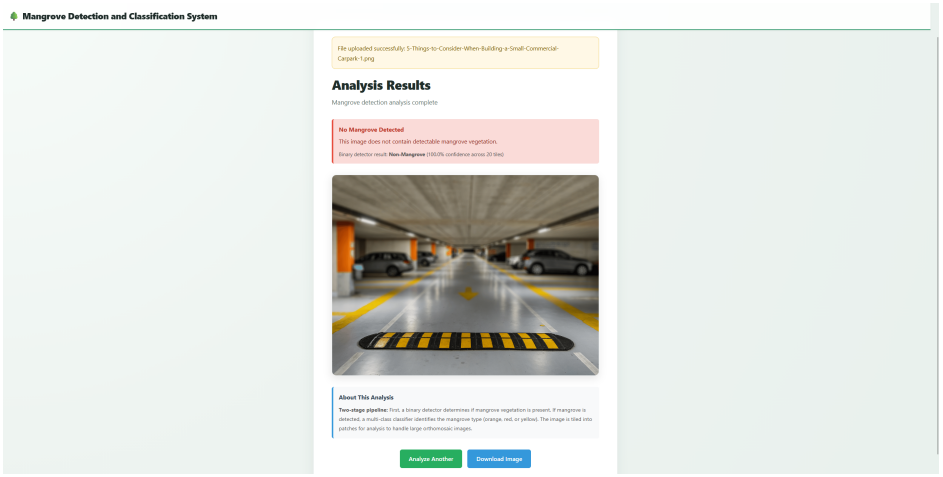


Figure 9 Web application interface indicates lack of mangroves within image

If the image does contain a mangrove, the page will display the predicted species, and its confidence in the prediction. An example can be seen in *Figure 10*.

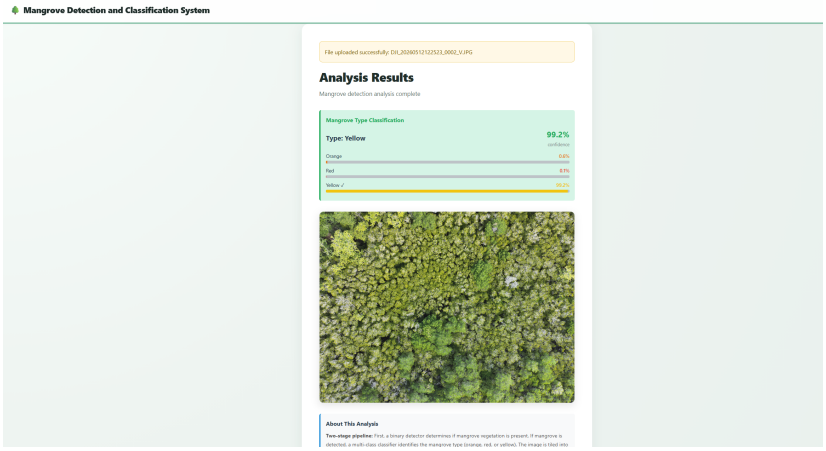


Figure 10 Web application interface indicates successful identification of mangroves as well as species shown in image

There are two buttons present at the bottom of this page, ‘Analyze Another’ redirects the user back to the home page where they can upload another image, ‘Download Image’ downloads the image shown on the page to the user's computer. The download image button currently does not have a useful purpose, the reason that it is there is because we had intended to provide the user with an overlay showing how many of each species were present in the image, but we ran out of time to properly implement this. An example of the object detection model using the DeepForest library [12] can be seen in *Figure 11*.

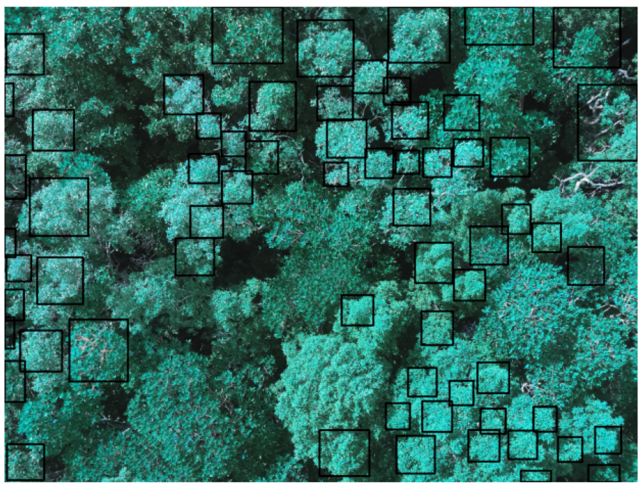


Figure 11 Unsuccessful crown detection model; blue-shifted image from processing, as well as decreased image quality and poor boxing.

We had hoped to have been able to show this on the results analysis page instead of the original image, however time constraints and real-world interruptions prevented us from being able to get this working in time for the project deadline, as mentioned previously.

Model and Flask App Integration

To integrate our model into a Flask web app, we restructured our Jupyter notebooks into standalone Python files. This involved removing visualisation and debugging code, focusing solely on prediction logic. We replaced absolute file paths with relative paths using ‘pathlib’ for portability.

Key to integration was creating functions callable by the Flask app: ‘build_model’ (constructs the model), ‘load_model’ & ‘load_binary_model’ (load saved models, raising errors if missing). These support image classification functions that return predictions and confidence scores.

We also implemented optimisations like dynamically detecting class names from directory structure – simplifying additions of new classes – and a standardisation function to convert incoming images (PNG, JPG/JPEG, TIFF) into a consistent format for the model.

Finally, we combined our binary (mangrove present/absent) and multi-class (species identification) models into a single 'predict_combined' pipeline. This first uses the binary classifier; if mangroves are detected, it then passes the image to the multi-class classifier for species prediction, delivering results to the user interface.

Baseline Random Forest Binary Classification Model Evaluation

The binary random forest classification model achieved 93.75% accuracy on the test dataset, correctly classifying the dataset samples as either mangrove or non-mangrove. The model metrics indicated strong performance, however distinct disparities were observed from the results.

Firstly, the model accurately predicted the presence of mangroves with 100% precision (*Table 3*), however missed 29% of real mangrove images resulting in a 71.43% recall. This indicates that the model's generalisation on the given input data may not be sufficient.

In contrast, the model was able to predict all non-mangrove images with a 100% recall however missed 7.5% testing data with precision. Due to the nature of the training data sources, the model was developed using 7-band satellite imagery of mangroves as temporary solution until AI First Nations mangrove captured data could be provided. When the model was prompted to predict on AI First Nations captured data, the model failed to generalise relationships and incorrectly predicted every label.

This can be explained due to the spatial difference between the AI-First Nation's drone data and the temporary Kaggle satellite data [4] containing different dimensions, imagery bands and image compositions. The inability for the model to generalise features prompted the development of an improved model utilising new architecture to more accurately capture the provided drone dataset.

Table 3 Precision, recall, F1-score, and support for Random Forest Binary Classifier

	precision	recall	f1score	support
Non-Mangrove	0.9259	1.0000	0.9165	25
Mangrove	1.0000	0.7143	0.8333	7

Table 4 Accuracy, macro-average, and weighted average for Random Forest Binary Classifier

accuracy			0.9375	32
macro avg	0.9630	0.8571	0.8974	32
weighted avg	0.9421	0.9375	0.9335	32

Advanced Custom Neural Network Binary Classification Model Evaluation

The advanced custom neural network binary classifier used EfficientNet as the backbone, achieving an accuracy of 98.96 on the test dataset. The advanced model's evaluation metrics are provide stronger metrics than the base line model (random forest) with an improved accuracy of 98.86%. This highlighted a 5.61% improvement. Further, the model demonstrated stronger generalisation and predictability as seen with the results for each metric, except for precision and recall for non-mangrove and mangrove.

Firstly, the non-mangrove label resulted in a precision of 98.69%, recall of 98.69 and f1 score of 98.61%. This indicated that the advanced model adequately classified the testing data more effectively than the baseline model for non-mangroves. Improvements were also present for mangrove identification as seen with the precision of 99.12%, recall of 99.21% and 99.17% f1-score.

Most notable from the results obtained was the 20.88% increase to recall of mangrove images in comparison to the baseline models results.

These results could be attributed to the high-quality imagery used in the training, testing and validation. The model's dataset displayed an imbalance in the type of data used to train the non-mangrove class. Previously, the model generalised features on 7455 tiled images of non-mangrove forests prompting the model to generalise and extract incorrect features on canopies, RGB values, contrast and brightness. By providing 6007 additional diverse input data, the model trained effectively and efficiently on other sceneries creating new neural network features leading to the improvement of the model at large.

Table 5 Precision, recall, F1-score, and support for EfficientNet CNN classifier

	precision	recall	f1score	support
Non-Mangrove	0.9869	0.9854	0.9861	1987
Mangrove	0.9912	0.9921	0.9917	3299

Table 6 Accuracy, macro-average, and weighted average for EfficientNet CNN classifier

accuracy			0.9896	5286
macro avg	0.9891	0.9888	0.9889	5286
weighted avg	0.9896	0.9896	0.9896	5286

Furthermore, when the advanced model was tested on predicting unseen data from our industry partner, the model perfectly predicted every image label (Table 7). This highlighted the advanced CNN model as a superior and balanced model compared to the base line random forest classifier.

Table 7 Results on unseen data

Non-Mangrove	17 (25%)
Mangrove	51 (75%)

Advanced Custom Neural Network Image Classification Model Evaluation

The advanced Custom Neural Network Image classification model utilising EfficientNet as the backbone achieved a strong performance of 97% accuracy across the test dataset, correctly classifying the dataset samples as either orange, red or yellow. Furthermore, the model achieved an average precision 99% in correctly producing true positive when predicated against positive and an average recall of 99% of producing correctly labelled classes indicating the model is balanced in predicting the species of mangroves across all three types. However, analysing the classes, the red mangrove classes precision drops to 91%. This can be explained due to the mangrove dataset having the smallest dataset out of all the mangroves, the red mangrove class contains the data size of 618 tiled images, orange with 1248 tiled images and lastly yellow with 2222 tiled images. Even after data processing by including weights and data augmentation, the model may have not been able to fully generalise features. Both yellow and orange correctly predicted 99% of all testing images based on the F1-score however oranges precision dropped to 98% indicating the model predicted a false positive. The F1-score of 97% indicates that the model is balanced and performs strongly when generalising feature of species despite the class imbalance.

Table 8 Precision, recall, F1-score, and support for Species Classifier

	precision	recall	f1score	support
Orange	0.98	0.97	0.87	1248
Red	0.91	0.97	0.94	610

Yellow	0.99	0.97	0.98	2222

Table 9 Accuracy, macro-average, and weighted average for Species Classifier

accuracy			0.97	32
macro avg	0.96	0.97	0.97	32
weighted avg	0.97	0.97	0.97	32

Lastly, the advanced Custom Neural Network Image classification model successfully predicted and labelled all unseen testing data. Which viewed with the confusion matrix, as all images are classified correctly in their respected labelled.

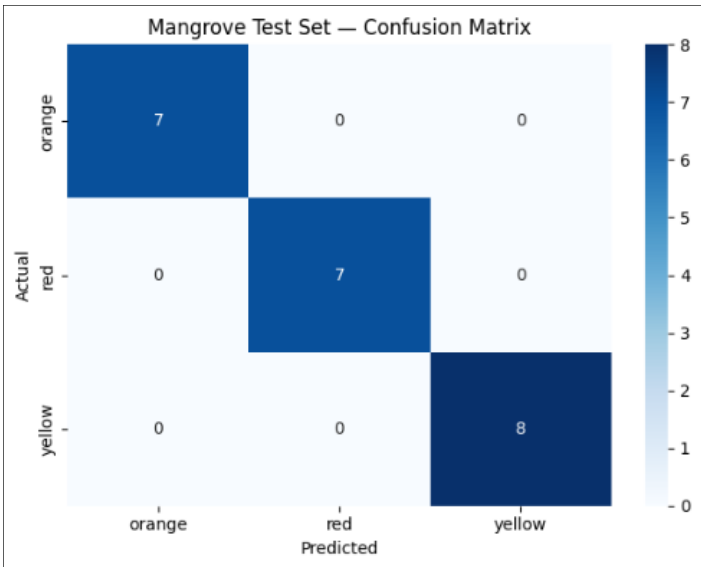


Figure 12 Confusion Matrix for Species Classifier

Results and Findings

An analysis of research into the datasets and machine learning models revealed for the Custom Neural Network Image classification model key features that assists in the differentiation of the species. We found by analysing the RGB colour bands, the red mean intensity highlighted an inverse relationship in generalising features. Yellow mangrove species displayed the highest red intensity mean of 112, orange mangrove species displayed a red intensity mean of 109 and r displayed a red intensity mean of 99. The statistical meaning behind the red mean intensity is the difference in light absorption and light reflection on the mangrove's species using AI First Nations imagery. This indicates that each mangrove species has a distinct red-light absorption in the leaf pigmentation and canopy structure that is captured in imagery.

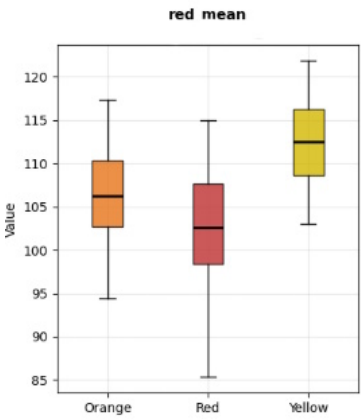


Figure 13 Red light absorption by species

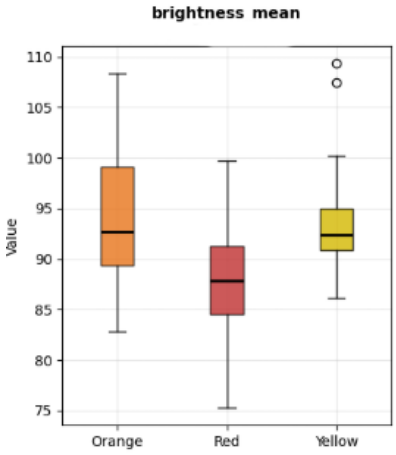


Figure 14 Brightness features by species

Continuing the next important finding was the average brightness of the mangroves with orange being 7, yellow being 94 and red being 84. The average brightness value represents how the luminance in the image reflects off canopies and vegetation within the images. By pairing this information with the red RGB mean index to map out sections of scenery by reflections, depth and features for the machine model to learn.

Lastly, the feature of canopy coverage indicated the largest statistical indictor of differentiating species within the dataset. This was done by identifying and counting which pixels in the images were green dominated. The total sum of green pixels in an image divided by total pixels created a mask and statistical evidence of the proportion of an image is green or vegetation.

Concluding the integration of exploratory data analysis, pre-processing and data augmentation throughout advanced models pipeline provided the grounding for efficient and effective models to be developed evident from our evaluation metrics.

Final deliverable Reflection

Given that we are handing this project off to other developers at the conclusion of the semester, we made sure to include documentation that will help future users and developers of the model understand the code and how everything works. This includes a readme file giving instructions on how to run the application, as well as helpful comments explaining details such as how elements in the code run, the purpose of them. This documentation will allow current and future users of the model to easily understand how to use and develop the models further.

Conclusion

Our team successfully delivered two machine learning models that utilise neural networks that can both detect and classify mangroves from drone and aerial imagery, a functional user interface that allows users to upload their own aerial mangrove photos and receive automatic species classification. We laid the groundwork for future development on this technology despite setbacks in data acquisition, and are handing off a functional, well documented repository for those who come after.

The members of our team gained valuable data science and machine learning skills, we learnt how to work collaboratively in a team. The skills and experience we gained through this project will be invaluable as we continue to advance through our careers. We hope that the technology developed will be useful for AI First Nation’s mission and that the models we developed improve environmental monitoring.

References

[1] R. Mahajan, 2026. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/f1-score-in-machine-learning/. [Accessed 4 June 2026].

[2] GeeksForGeeks, “Precision, Recall and F1-Score using R,” GeeksForGeeks, 9 April 2025. [Online]. Available: https://www.geeksforgeeks.org/r-language/precision-recall-and-f1-score-using-r/. [Accessed 4 June 2026].

[3] Google, “Classification: Accuracy, recall, precision, and related metrics,” Google, [Online]. Available: https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall. [Accessed 4 June 2026].

[4] S. Sharma, “Mangrove Satellite Distinguishing,” February 2026. [Online]. Available: https://www.kaggle.com/datasets/saanujsharma/mangorve-distinguishing/data.

[5] VisDrone, “VisDrone-Dataset,” 2023. [Online]. Available: https://github.com/VisDrone/VisDrone-Dataset. [Accessed 4 June 2026].

[6] D. Joseph, “American Red Cross projects,” July 2018. [Online]. Available: https://community.opendronemap.org/t/american-red-cross-projects/806. [Accessed 4 June 2026].

[7] A. Bhardwaj and Y. Tuteja, “Aerial Landscape Images,” [Online]. Available: https://www.kaggle.com/datasets/ankit1743/skyview-an-aerial-landscape-dataset?resource=download. [Accessed 4 June 2026].

[8] P. Sakthi U R, “Multilabel-GeoSceneNet-16K,” [Online]. Available: https://www.kaggle.com/datasets/prithivsakthiur/multilabel-geoscenenet-16k. [Accessed 4 June 2026].

[9] L. Raju, “Intel Image Classification Dataset,” [Online]. Available: Intel Image Classification Dataset. [Accessed 4 June 2026].

[10] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” *International Conference on Machine Learning, 2019,* 2019.

[11] A. Lee, “Adam vs SGD : What are the optimizers in neural network and when do we use?,” Medium, 31 May 2024. [Online]. Available: https://medium.com/@pumadd1227/adam-vs-sgd-what-are-the-optimizers-in-neural-network-and-when-do-we-use-238478a0eaea. [Accessed 2 June 2026].

[12] The Environmental AI community, “Tree Crown,” The Environmental AI community, 2021. [Online]. Available: https://acocac.github.io/environmental-ai-book/forest/modelling/forest-modelling-treecrown_deepforest.html. [Accessed 2 June 2026].

Appendices

Code and Project Management

Code was managed using the version control system, GitHub. This enabled streamlined and independent work through the course of the project. An atomic-commit methodology was used through the course of the project, meaning that all changes to the model can be tracked.

The project was managed using a Trello dashboard, adapting the Kanban cards-based methodology to assign tasks, communicate with project supervisors, and share resources from the web. Communication was coordinated through our group's Microsoft Teams page, as well as Discord.

Weekly meetings were organised with both academic and industry-partner supervisors to ensure open communication, especially as the aim of the project shifted with time.

Roles and Responsibilities

To stay on track with the project, the three of us took on dedicated roles and responsibilities so that we could efficiently delegate tasks to one another.

Kirtan took on the role of organising weekly meetings with our academic and industry supervisors, and spearheaded communication with them, providing regular updates on our progress, and letting them know if we were running into difficulties. He also developed a prototype of the tree counting system and assisted the team with development.

Jake took notes in the weekly meetings and compiled them into a shared group chat, allowing for easy access to the latest requirements or requests from either the academic or industry supervisor. Jake also took on the role of model development, before receiving the data from AI First Nations he developed prototype models based on the Kaggle dataset, and after receiving the data developed two neural network models.

Tom developed the Flask application and integrated the machine learning models developed by Jake into the web hosted interface, he also took on the role of managing the Trello board, updating it with questions for the academic and industry supervisor, adding relevant resources and keeping track of project progress.

Given the nature of the task, we thought it best to delegate specific tasks, which meant that a lot of time was spent working alone. Despite this, we took care to always keep each other in the loop, and as a result we were able to complete the revised deliverables incredibly quickly once the data was received.

Self-reflection

Kirtan

This project was an incredibly enlightening and exciting project. It saw us, at the start of the semester with many ideas, all whittled down into one refined one very quickly. I really admired the ability of my peers in this project to quickly action ideas and create prototypes, despite a lack of resources for much of it. This demonstrated a resourcefulness and a desire to present even an inkling of a workable product within a short amount of time.

Working through this project taught a lot about working with Indigenous stakeholders and working in partnership with Indigenous people. I felt that this was especially important in this country.

I feel that working in an Agile methodology came quite naturally for the scope of the project and really enabled us to refine the deliverables as we went. Support from both Damiano and John really made this project approachable and exciting to do, while also maintaining accountability for us in the team.

Going forward, I believe that my approach to this project was a little bit hands-off, which gave rise to very little experimentation on my end. This meant that very little machine learning and training was

conducted by me, leading to less personal skill development. In future, I must not be afraid to get my hands dirty.

Thomas

Throughout this course I learnt a lot about productive time management, in times where there may appear to be nothing to do finding some way to contribute positively to the overall project was a skill I developed over the course of the semester. I believe we worked well together as a team and communicated effectively as a team, if I think back to what I could have done better it would be better communication with my team in regards to my schedule, there were many times where I was unable to attend meetings because I failed to notify my team of my work schedules before they were organised. I believe that the team allocated tasks effectively and communicated our progress on these tasks well, we were able to overcome a late start due to factors outside of our control and still deliver something I can be proud of.

Jake

Throughout my time within this course, I learnt a lot about what real world production can and feel like. Learning to properly and adequately communicate with industry and academic supervisors whilst being part of a group taught me lots of lessons of time management and cooperation. Based on the circumstances that my team and I were provided, I believe we navigated the situation quite positively with having a group member leave and complications and receiving data. Regarding myself, there were times I was unable to attend meetings due to unforeseen important circumstances, however i was able to effectively communicate with my team. The team allocated the tasks well between each other and communicate the tasks effectively. If I was to redo this project, I would effectively communicate more with my industry partner about receiving data however my team and I were able to overcome this.